Java Programming 1
Lecture 4
Objects and Methods

Using Object Methods,
Reading the Java API Documentation
Introducing Karel's World

Express Yourself

- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic04
  - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 4"
- **Exercise 1**: Open BlueJ and create a new project called ic04. Snap me a picture of your empty project

Object Variable Review

- For every **object** **variable** you describe:
  - What specific **kind** of thing the variable points to
  - What **name** you want to use for the variable

```
Color hairColor;
```

Construction Parameters

- Use **assignment** to specify the **object** it refers to
  - You can create the object itself in several different ways

```
hairColor = new Color(128, 255, 128);
hairColor = new Color(.25F, .75F, 1.0F);
hairColor = Color.gray;
hairColor = someButton.getBackground();
```

What are Parameters?

- In the last lecture, you learned to construct a **Circle** object, and send a message to it, like this:

```
Circle c = new Circle();   // variable and object
c.makeVisible();           // send a message
```

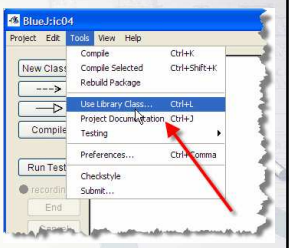
  - Note that we don't supply any additional information
- **Most** methods or constructors need extra information
  - A square root method needs the number to operator on
- Sent to your method via "**parameters**" or "**arguments**"

Kinds of Parameters

- There are two kinds of parameters:
  - **Formal parameters** are variables used in the method
    - Designed specifically to for passing information
    - We'll learn about these in Chapter 3
  - **Actual parameters** or **arguments** are the values that are supplied when the method or constructor is used
- We'll learn how to use actual parameters by creating some different objects from the Java Class Libraries
  - Rectangles, Colors, and Strings

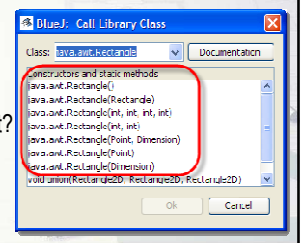
The Rectangle Class

- Let's start by examining the **Rectangle** class in BlueJ
- Choose **Use Library Class** from **Tools** menu
- Type full name, **java.awt.Rectangle**, press **Enter**
- **Exercise 2**: Snap a picture of the dialog



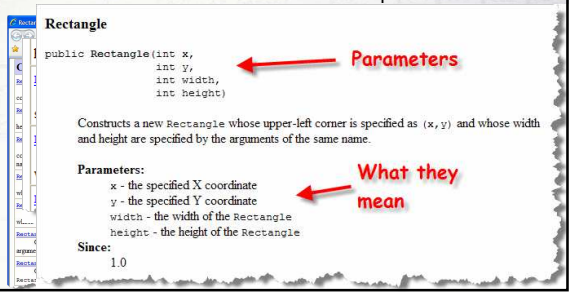
Different Constructors

- Notice that there are seven **different** versions of the **Rectangle** constructor (*overloaded constructors*)
  - Each requires a different type or number of values
- 1 with no parameters
- 4 with object parameters
- 2 with **int** parameters
- What do those represent?
- Click the **Documentation** Button



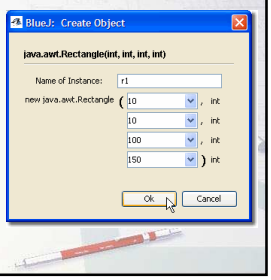
Reading the JavaDocs

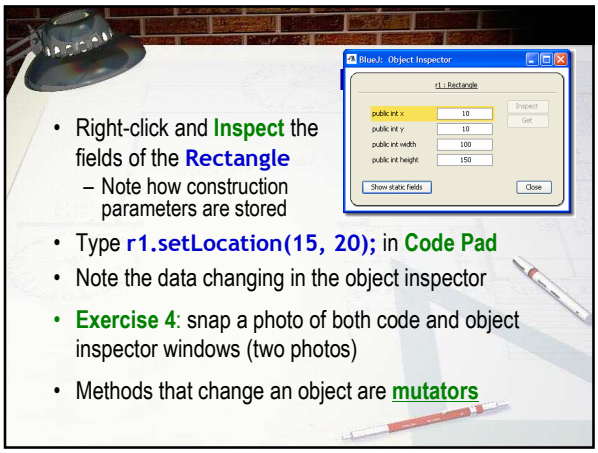
- Opens your browser to a screen showing more information about each constructor's parameters



Creating a Rectangle

- Choose the four-int constructor, then, supply a name and construction parameters
  - I've used the name **r1** and created a **Rectangle** that's 100 units wide and 150 high
- **Exercise 3**: snap a photo of your object on the object bench





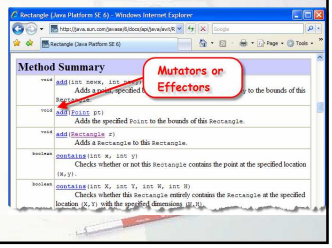
- Right-click and **Inspect** the fields of the **Rectangle**
  - Note how construction parameters are stored
- Type **r1.setLocation(15, 20);** in **Code Pad**
- Note the data changing in the object inspector
- **Exercise 4**: snap a photo of both code and object inspector windows (two photos)
- Methods that change an object are **mutators**

Mutators and Accessors

- Some methods. like **println()**, just **perform an action**
  - These simple methods are sometimes called **effectors**
- Some methods modify an object's internal state
  - These are called **mutator** methods
  - Changing the location of a **Rectangle**
- Some methods **return information about** an object
  - These kinds of methods are called **accessors**
  - The length of a **String** or the width of a **Rectangle**
- Some methods just **perform a calculation**

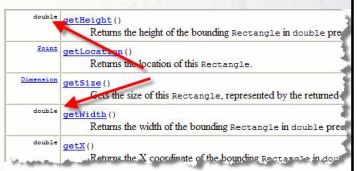
Identifying Mutators

- The documentation for effectors and for mutators start with the keyword **void** in the left-hand column
  - **System.out.println()** and **Rectangle's setLocation()** are **void** methods
- Locate the Methods section in the **Rectangle** documentation
- **Exercise 5**: call another mutator, snap a pic of both



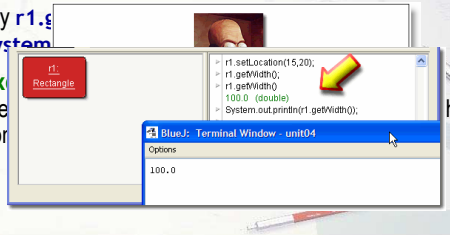
Accessors and Calculators

- Accessors and calculating methods are often called "**value-producing**" methods
  - Can use them wherever you'd use a **value**
  - Called **functions** in languages like VB and Pascal
- Have **return type** instead of **void** in the signature



Rectangle Accessors

- Use methods to examine the state of the object **r1**
  - **getX()**, **getY()**, **getWidth()**, **getHeight()**
- Type **r1.getWidth();** in Code Pad
- Try **r1.g**
  System
- **Ex**
  me he
  cor



Fully-qualified Names

- **Exercise 7**: in Code Pad, try to create a **Rectangle** variable named **r2**. Snap a picture of the result.
- The **Rectangle** class is in a **library** or **package**
  - The package is named **java.awt**
  - To define a variable, you use a **fully-qualified** name

```
java.awt.Rectangle r2;  // variable and object
```

- **Exercise 8**: in Code Pad, use the fully-qualified name to create a **Rectangle** variable named **r2** and to initialize it to the dimensions 1, 1, 50, 75. Snap a picture of the code you wrote and inspect the object.

Using import

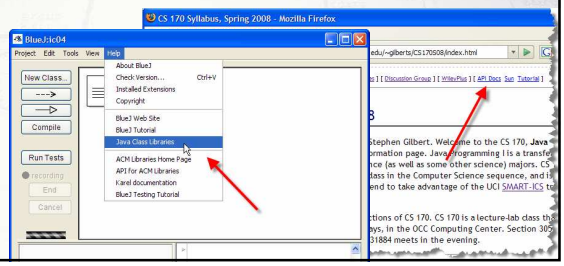
- Why not use the full name (**java.awt.Rectangle**)?
  - It's a lot of work and it clutters up your code
- Instead, normally use the **import** statement

```
import java.awt.Rectangle;
import java.awt.*;
```

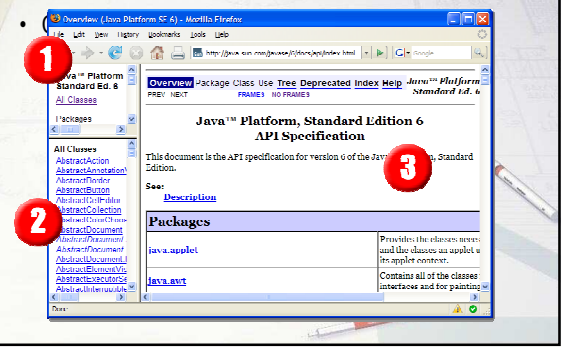
  - Second version allows access to all classes in package
  - This must appear before the class definition
- **Exercise 9**: in Code Pad, use import and then make a a **Rectangle** variable named **r3**. Snap a picture of the code you wrote and inspect the object.

More Documentation

- Both the class homepage and BlueJ have links to the Java API docs (instead of "Use Library Class")

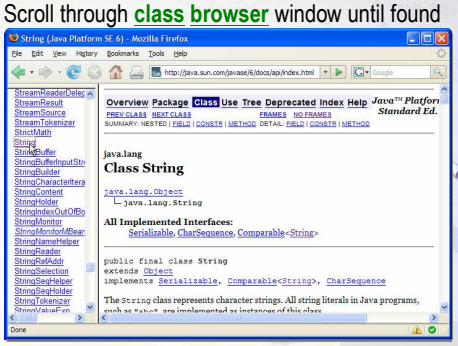


The JavaDoc API Window



Finding the String Class

- Scroll through **class browser** window until found

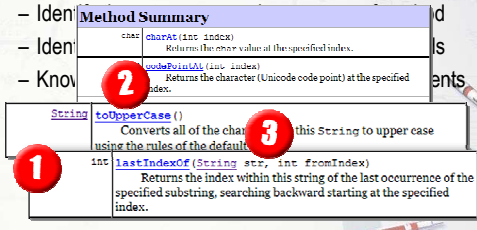


Class Summary

- Locate the inheritance tree and the package name
  - Use the package in your **import** statement
  - The **java.lang** package is automatically imported
- Read the class summary for an overview of the class

Deciphering the Docs

- Read method documentation to “talk” to your objects
  - Locate the method summary
  - Iden
  - Iden
  - Know



Explain Yourself

- Using **only** what you've learned up to now in class:
- **Exercise 10**: What **import** statement would you have to add to your applet if you wanted to use the **TimeZone** class? How or where would you find the correct import? Shoot a screenshot showing me.
- **Exercise 11**: What **mutator** methods could you call, assuming you only knew how to pass **String**, **int** and **double** parameters?

Explain Yourself

- **Exercise 12**: If your program contains a **TimeZone** object named **tz**, write the statements to call each of the methods from the previous question. The actual values you use are unimportant if you use the right "kind" of thing.

Using Returned Values

- **Ignore** it: "legal", but not useful. **Don't do this**!

```
name.toUpperCase();
```

- **Store** it in a variable? Variable must be correct type!

```
String bigName = name.toUpperCase();
```

- **Use** the "value produced" directly like this:

```
System.out.println(name.toUpperCase());
```

String Length

- How many chars in a **String**? Use **length()** method

```
1: String s1 = "Conch", s2 = "fritter", s3 = "", s4;
2: int len1 = s1.length();
3: System.out.println("s2 has " + s2.length() + "chars");
4: s3.length();
5: System.out.println("s4.length() is " + s4.length());
```

1. Four **String** variables, 3 initialized with literals
2. Call method, store answer (5) in variable
3. Use method as a value along with concatenation
4. Call method, but don't use value (**logic error**)
5. Attempt to send message to an **uninitialized object**

Express Yourself

- Let's put this all together
- Look up the docs for the **java.util.Date** class
  - Examine the constructor summary
  - **Ignore** the deprecated notes for right now
- **Exercise 13**: Create two **Date** objects in Code Pad
  - Have one represent the current time (**now**)
  - Have one represent your birthday (**bday**)
  - Shoot me a screen-shot of Code Pad

Express Yourself

- **Exercise 14**: Examine the method summary section
  - How many methods are accessor methods?
  - How many methods are mutator methods?
  - How many are effectors or carry out a calculation?
- **Exercise 15**: Use some **Date** methods
  - Change the variable **now** to refer to this time yesterday
  - Print **bday** in the form 2/2/1950 using **System.out**
  - Print **now** as "words" using **System.out**
  - Shoot a screen-shot of Code Pad and another of the console window with your output.

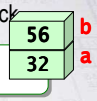
How Programs Store Data

- 3 Types of data storage space (at program run time)
  - Static storage area
    - Data loaded into memory when your program starts
    - Memory is reserved for as long as your program runs
  - Stack-based storage area
    - Short-lived data used for the duration of a single method
    - Memory can then be reused by other variables
  - Heap or "free-store"-based storage area
    - Dynamic, long-lived reference types

Value and Reference Types

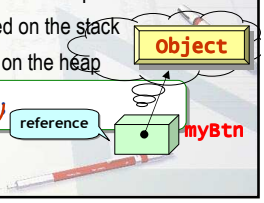
- Value types store all of their data on the stack

```
int a = 32, b = 56;
```

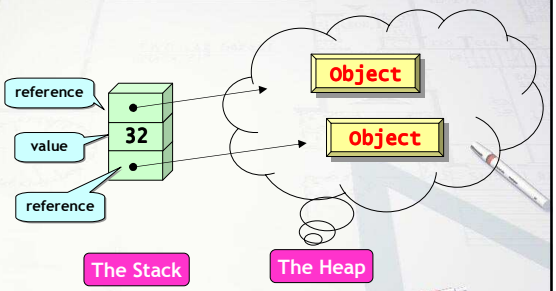


- Reference types are divided into two pieces
  - The reference variable stored on the stack
  - The unnamed object stored on the heap

```
Button myBtn = new Button();
```



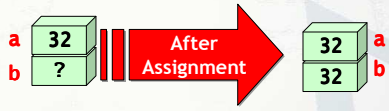
The Stack and the Heap



Assignment and Value Types

- Value-type vars are "buckets" holding **actual values**
  - Assignment makes duplicate, **independent** copy

```
int a = 32, b;          // Two variables
b = a;                  // Two independent values
```

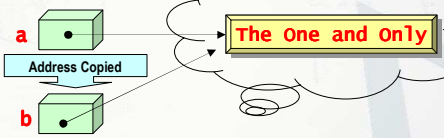


- Often called a **deep copy**

Reference Type Assignment

- Copying reference types, only the **address** is copied

```
Button a = new Button();
a.setText("The One and Only");
Button b = a;
```



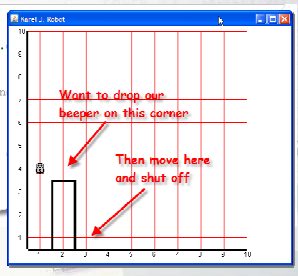
Karel Revisited

- Open the ic03 project, create a robot, call **task()**.
  - Here's code to pick up the beeper, move to 4th and 1st

```
public void task()
{
   World.startFromURL("http://csjava.
   karel = new OCRobot();

   // Pick up the beeper on 2nd St an
   karel.move();
   karel.pickBeeper();
   karel.move();
   karel.move();    // now on corner
   karel.turnOff();

}
```



Thinking like a Programmer

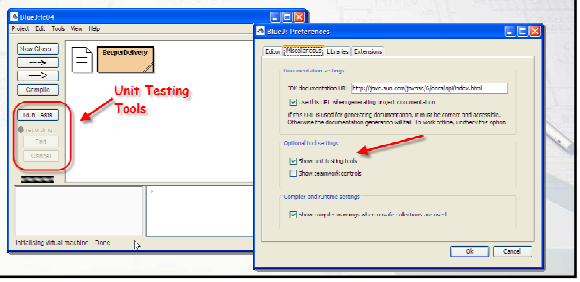
- We need to turn right, but Karel doesn't know how
  - Turn simple commands into more complex ones
  - Use a **sequence** of simpler commands
- Turn right by turning left three times
- **Exercise 16**: Complete the commands to drop Karel's beeper at 4th St. and 2nd Ave., and then continue on to 1st St. and 3rd Ave. before he turns himself off. Shoot two screen shots, one of your finished code and one of the completed task.

How Did You Do?

- How do you know you did the assignment correctly?
  - You check the **actual** results against the **expected**
- What are the expected results?
  - Karel should be on 1st Street and 3rd Avenue
  - Karel should be turned off
  - Karel should have no beepers in his beeper bag
  - The world should contain only a single beeper located at 4th Street and 2nd Avenue
- We can test program correctness by running a **unit testing** tool called **JUnit**, built in to BlueJ

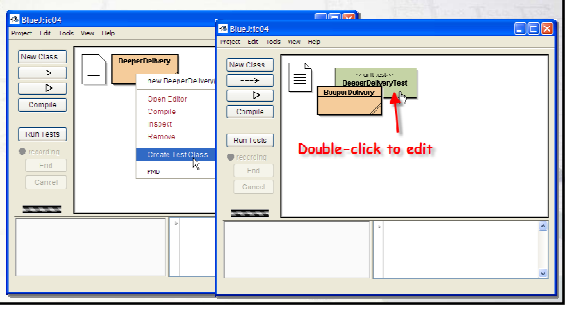
Unit Testing Tools

- Make sure that the Unit Testing Tools are displayed
  - If not, turn them on by using the Tools->Preferences



Creating a Test Class

- Right-click **BeeperDelivery** and **Create Test Class**

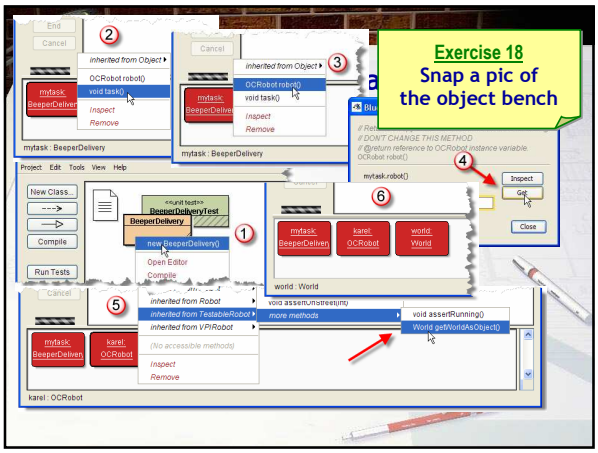


Complete the Documentation

- **Exercise 17**: Complete the heading at the top of the new test class and shoot two screen shots, one of your code and one of the window with the test class.

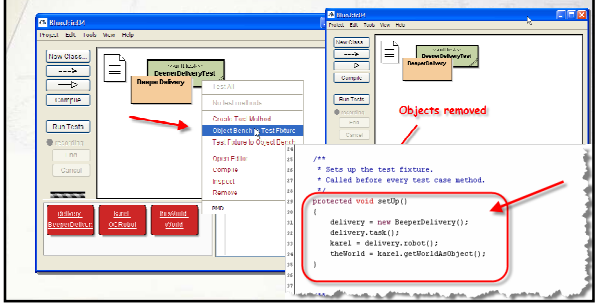
Create a Fixture

- We need to run our task before we can see if it worked
  - The completed situation that we'll test is our **fixture**
  - It will be recreated before every test is executed
- Create the test fixture like this:
  - Create an instance of your **BeeperDelivery** class
  - Invoke it's **task()** method (just like you did before)
  - Once the task is complete, invoke the **robot()** method
    - When the Method Result dialog appears, click **Get**
  - Invoke the robot's **getWorldAsObject** method and Get
    - Inherited from TestableRobot, more methods



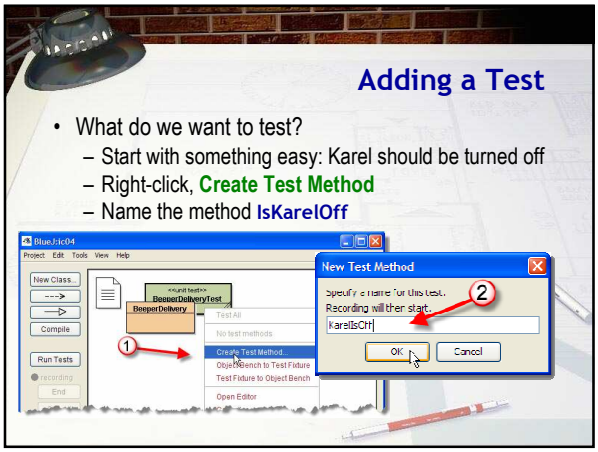
Create the Fixture

- Right-click test class: **Object Bench to Test Fixture**



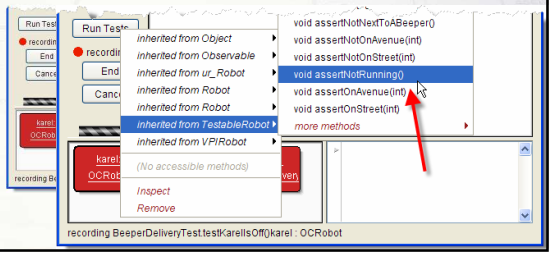
Adding a Test

- What do we want to test?
  - Start with something easy: Karel should be turned off
  - Right-click, **Create Test Method**
  - Name the method **IsKarelOff**



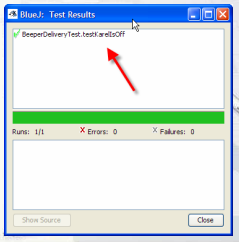
Recording Results

- Task run, object bench populated, recorder started
  - Right-click robot, choose **assertNotRunning()**
  - Click the **End** button to stop the recorder



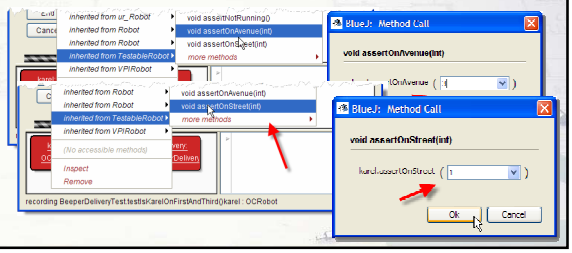
Assertions

- Assertions are statements about the **expected** result
  - The **TestableRobot** class has many different ones
  - **assertNotRunning** means we expect karel to be off
- Running the unit tests
  - Click **Run Tests** button
  - Running tests are **green**
- **Exercise 19**: Show me your test running green



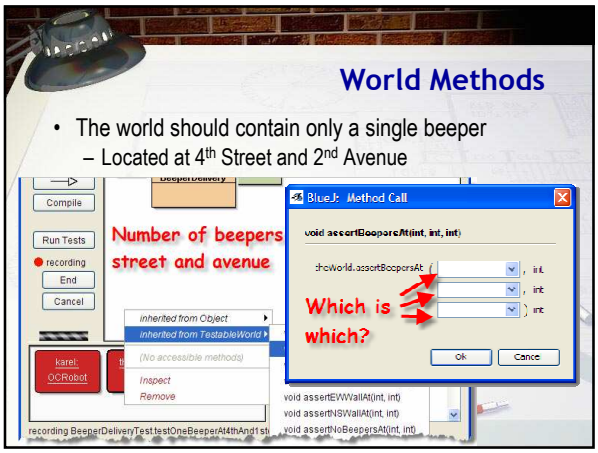
More Tests

- Let's write tests for remaining requirements
  - Karel's BeeperBag should be empty
  - Karel should be on 1st Street and 3rd Avenue

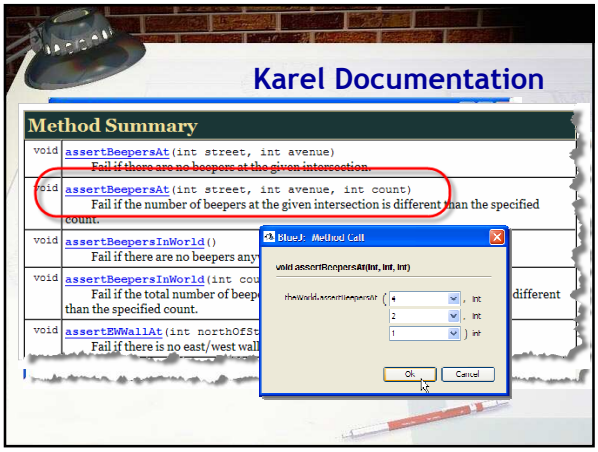


World Methods

- The world should contain only a single beeper
  - Located at 4th Street and 2nd Avenue

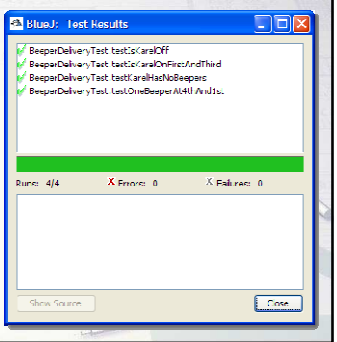


Karel Documentation



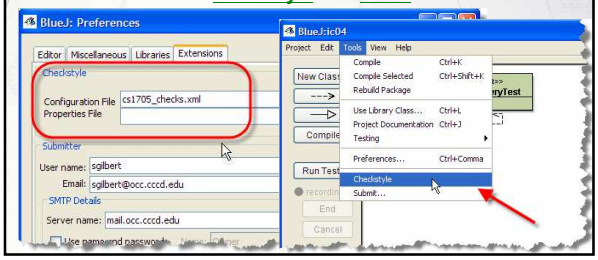
Express Yourself

- **Exercise 20**: Show me all four tests running green



Checkstyle Revisited

- Checkstyle automatically flags **style** errors
  - Configuration file: **cs1705_checks.xml**
  - Run: choose **Checkstyle** from **Tools** menu



Express Yourself

- **Exercise 21**: Use Checkstyle to make sure that your BeeperDelivery class meets the CS 170 style requirements. Shoot me a screenshot with a clean Checkstyle window displayed.
- In-class exercise: submit **before** you leave today!
  - Save as a PDF; drop into submissions folder on Q:
- Finish reading Chapter 2 (just skim graphics)
  - Chapter 2 quiz next Thursday (no graphics)
- Homework 5: A Larger Karel problem
  - Picking up eight beepers; turn in a PDF file on Q: drive